



Podstawowe polecenia SWI-Prolog

listing/1 — wyświetlenie definicji predykatu

Predykat listing/1 wypisuje definicję wskazanego predykatu. Można podać nazwę z arnością, np. rodzic/2; istnieje też listing/0, które wypisuje cały program.

Przykład programu

```
ojciec(jan, piotr).  
matka(anna, piotr).  
rodzic(X, Y) :- ojciec(X, Y).  
rodzic(X, Y) :- matka(X, Y).
```

Zapytanie

```
?- listing(rodzic/2).
```

Prolog wypisze definicję:

```
rodzic(X, Y) :-  
    ojciec(X, Y).  
rodzic(X, Y) :-  
    matka(X, Y).
```

Do czego to służy

To polecenie jest bardzo wygodne, gdy chcesz sprawdzić:

- czy predykat został poprawnie zapisany,
- ile ma klauzul,
- jak dokładnie wygląda jego definicja.

is/2 — obliczenia arytmetyczne

is/2 oblicza wartość wyrażenia arytmetycznego po prawej stronie i unifikuje wynik z lewą stroną. Typowo po lewej stronie stoi niezwiązana zmienna. Gdy chcesz **porównać** wartości liczbowe, do testu równości należy używać raczej `=:=/2`, a nie `is/2`.

Przykłady

```
?- X is 2 + 3.  
X = 5.  
?- Y is 4 * 5.  
Y = 20.  
?- Z is (10 - 2) / 4.  
Z = 2.
```

Przykład

```
pole_prostokata(A, B, P) :-  
    P is A * B.
```

Zapytanie:

```
?- pole_prostokata(4, 6, P).  
P = 24.
```

format/2 — estetyczne wypisywanie tekstu

format/2 służy do formatowanego wypisywania tekstu. Pierwszy argument to wzorec tekstu, a drugi to lista argumentów podstawianych do wzorca.

Prosty

```
?- format('Witaj.~n', []).
```

Tutaj `~n` oznacza nową linię.

Przykład z podstawianiem wartości

```
?- X = anna, format('Student: ~w~n', [X]).
```

Wynik:

```
Student: anna  
X = anna.
```

Przykład praktyczny

```
pokaz_studenta(Imie, Rok) :-  
    format('Student ~w jest na roku ~w.~n', [Imie, Rok]).
```

Zapytanie:

```
?- pokaz_studenta(ewa, 2).
```

Wynik:

```
Student ewa jest na roku 2.  
true.
```

Znacznik	Znaczenie	Przykład
<code>~w</code>	wypisuje term w zwykłej postaci	<code>format('~w', [ala]).</code>
<code>~q</code>	wypisuje term w postaci zgodnej ze składnią Prologa	<code>format('~q', ['Ala ma kota']).</code>
<code>~d</code>	wypisuje liczbę całkowitą dziesiętnie	<code>format('~d', [123]).</code>
<code>~f</code>	wypisuje liczbę zmiennoprzecinkową	<code>format('~f', [3.14]).</code>
<code>~n</code>	nowa linia	<code>format('Ala~nKot', []).</code>
<code>~c</code>	wypisuje znak o podanym kodzie ASCII/Unicode	<code>format('~c', [65]).</code>
<code>~s</code>	wypisuje napis jako listę kodów znaków	<code>format('~s', [[72,101,108,108,111]]).</code>
<code>~w</code>	wypisuje znak <code>~</code>	<code>format('~w', []).</code>

Do czego to się przydaje

format/2 jest wygodny, gdy:

- chcesz czytelnie wypisywać wyniki,
- stworzysz prosty interfejs tekstowy,
- budujesz raport z działania programu.

findall/3 — zebranie wszystkich rozwiązań do listy

findall/3 tworzy listę wszystkich podstawień, jakie otrzymuje wskazany wzorzec podczas przeszukiwania celu. Gdy cel nie ma rozwiązań, wynik to po prostu pusta lista [].

Przykład

```
student(ala).  
student(jan).  
student(ewa).
```

Zapytanie

```
?- findall(X, student(X), Lista).
```

Wynik

```
Lista = [ala, jan, ewa].
```

Jak to czytać

- X — co chcemy zbierać,
- student(X) — warunek wyszukiwania,
- Lista — wynik końcowy.

Drugi przykład

```
lubi(ala, matematyka).  
lubi(ola, matematyka).  
lubi(jan, informatyka).
```

Zapytanie:

```
?- findall(Osoba, lubi(Osoba, matematyka), Lista).
```

Wynik:

```
Lista = [ala, ola].
```

Gdy nie ma wyników

```
?- findall(X, student(piotr), Lista).  
Lista = [].
```

To także jest poprawne działanie findall/3.

include/1 w SWISH — dołączenie innego programu

W pomocy SWISH pokazano, że zapisany wcześniej program można wczytać do nowego programu za pomocą dyrektywy :- include(nazwa_programu). Można też dołączyć konkretną wersję programu po jej identyfikatorze.

Przykład

```
:- include(clever).
```

To oznacza: dołącz program zapisany wcześniej pod nazwą clever.

Kiedy to jest użyteczne

- gdy chcesz korzystać z wcześniej zapisanej bazy wiedzy,
- gdy dzielisz większy materiał na kilka plików,
- gdy prowadzący udostępnia gotowy moduł do wykorzystania na laboratorium.

Przykład łączący kilka poleceń

```
student(ala).
student(jan).
student(ewa).

liczba_studentow(N) :-
    findall(X, student(X), Lista),
    length(Lista, N).

pokaz_liczbe_studentow :-
    liczba_studentow(N),
    format('Liczba studentow: ~w~n', [N]).
```

Co można uruchomić

```
?- listing(student/1).
```

Wypisze definicje faktów student/1.

```
?- findall(X, student(X), Lista).
```

Zwróci listę wszystkich studentów.

```
?- pokaz_liczbe_studentow.
```

Wypisze komunikat z użyciem format/2.